TDS Archive

DATA DRIVEN GROWTH WITH PYTHON

Customer Segmentation

Segmentation by RFM clustering

Barış Karaman Follow 6 min read · May 4, 2019

Introduction

This series of articles was designed to explain how to use Python in a simplistic way to fuel your company's growth by applying the predictive approach to all your actions. It will be a combination of programming, data analysis, and machine learning.

I will cover all the topics in the following nine articles:

1- Know Your Metrics

2- Customer Segmentation

3- Customer Lifetime Value Prediction

4- Churn Prediction

5- Predicting Next Purchase Day

6- Predicting Sales

7- Market Response Models

8- Uplift Modeling

9- A/B Testing Design and Execution

Articles will have their own code snippets to make you easily apply them. If you are super new to programming, you can have a good introduction for Python and Pandas (a famous library that we will use on everything) here. But still without a coding introduction, you can learn the concepts, how to use your data and start generating value out of it:

> Sometimes you gotta run before you can walk — Tony Stark

As a pre-requisite, be sure Jupyter Notebook and Python are installed on your computer. The code snippets will run on Jupyter Notebook only.

Alright, let's start.

Part 2: Customer Segmentation

In the previous article, we have analyzed the major metrics for our online retail business. Now we know what and how to track by using Python. It's time to focus on customers and segment them.

But first off, why we do segmentation?

Because you can't treat every customer the same way with the same content, same channel, same importance. They will find another option which understands them better.

Customers who use your platform have different needs and they have their own different profile. Your should adapt your actions depending on that.

You can do many different segmentations according to what you are trying to achieve. If you want to increase retention rate, you can do a segmentation based on churn probability and take actions. But there are very common and useful segmentation methods as well. Now we are going to implement one of them to our business: **RFM.**

RFM stands for Recency - Frequency - Monetary Value. Theoretically we will have segments like below:

- **Low Value**: Customers who are less active than others, not very frequent buyer/visitor and generates very low - zero - maybe negative revenue.
- **Mid Value**: In the middle of everything. Often using our platform (but not as much as our High Values), fairly frequent and generates moderate revenue.
- **High Value**: The group we don't want to lose. High Revenue, Frequency and low Inactivity.

As the methodology, we need to calculate *Recency, Frequency and Monetary Value* (we will call it Revenue from now on) and apply *unsupervised machine learning* to identify different groups (clusters) for each. Let's jump into coding and see how to do **RFM Clustering**.

Recency

To calculate recency, we need to find out most recent purchase date of each customer and see how many days they are inactive for. After having no. of inactive days for each customer, we will apply K-means* clustering to assign customers a *recency score*.

For this example, we will continue using same dataset which can be found here. Before jumping into recency calculation, let's recap the data work we've done before.

```
# import libraries
from datetime import datetime, timedelta
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from __future__ import division

import plotly.plotly as py
import plotly.offline as pyoff
import plotly.graph_objs as go

#inititate Plotly
pyoff.init_notebook_mode()

#load our data from CSV
tx_data = pd.read_csv('data.csv')

#convert the string date field to datetime
tx_data['InvoiceDate'] = pd.to_datetime(tx_data['InvoiceDate'])

#we will be using only UK data
tx_uk = tx_data.query("Country=='United Kingdom'").reset_index(drop=True
```

g2_g1_recap.py hosted with ❤ by GitHub view raw

Now we can calculate recency:

```
#create a generic user dataframe to keep CustomerID and new segmentation sc
tx_user = pd.DataFrame(tx_data['CustomerID'].unique())
tx_user.columns = ['CustomerID']

#get the max purchase date for each customer and create a dataframe with it
tx_max_purchase = tx_uk.groupby('CustomerID').InvoiceDate.max().reset_in
tx_max_purchase.columns = ['CustomerID','MaxPurchaseDate']

#we take our observation point as the max invoice date in our dataset
tx_max_purchase['Recency'] = (tx_max_purchase['MaxPurchaseDate'].max() -

#merge this dataframe to our new user dataframe
tx_user = pd.merge(tx_user, tx_max_purchase[['CustomerID','Recency']], on=

tx_user.head()

#plot a recency histogram

plot_data = [
    go.Histogram(
        x=tx_user['Recency']
    )
]

plot_layout = go.Layout(
        title='Recency'
    )
fig = go.Figure(data=plot_data, layout=plot_layout)
pyoff.iplot(fig)

```

g2_calc_recency.py hosted with ❤ by GitHub view raw

Our new dataframe **tx_user** contains recency data now:

```
tx_user.head()
```

	CustomerID	Recency
0	17850.0	301
1	13047.0	31
2	13748.0	95
3	15100.0	329
4	15291.0	25

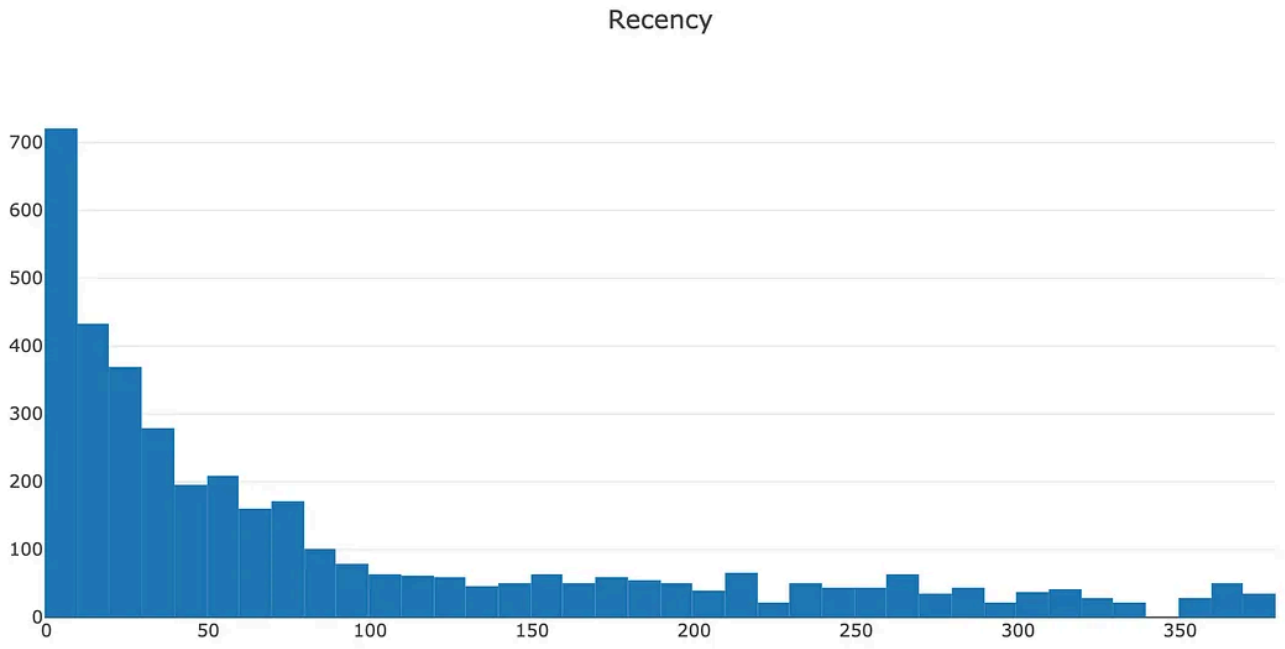
To get a snapshot about how recency looks like, we can use pandas' **.describe()** method. It shows mean, min, max, count and percentiles of our data.

```
tx_user.Recency.describe()
```

```
count    3950.000000
mean       90.778481
std       100.230349
min         0.000000
25%        16.000000
50%        49.000000
75%       142.000000
max       373.000000
Name: Recency, dtype: float64
```

We see that even though the average is 90 day recency, median is 49.

Our code snippet above has a histogram output to show us how is the distribution of recency across our customers.



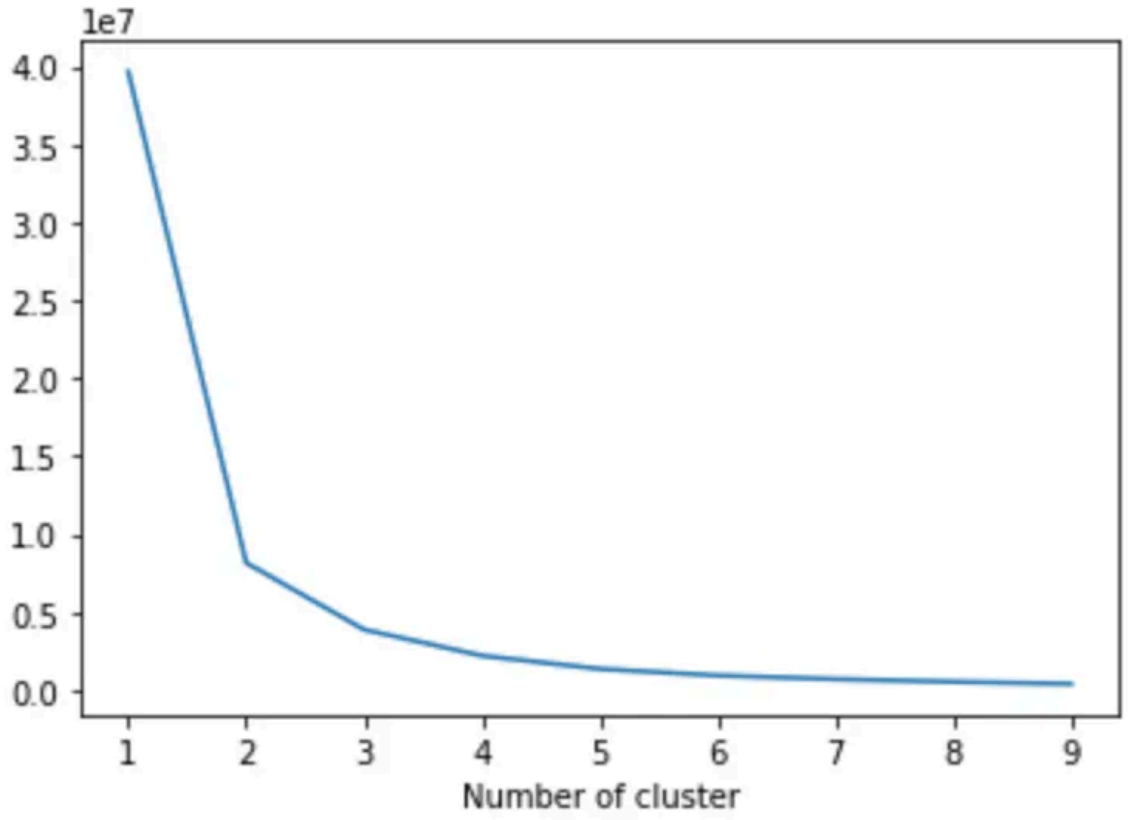
Now it is the fun part. We are going to apply K-means clustering to assign a recency score. But we should tell how many clusters we need to K-means algorithm. To find it out, we will apply Elbow Method. Elbow Method simply tells the optimal cluster number for optimal inertia. Code snippet and Inertia graph are as follows:

```
from sklearn.cluster import KMeans

sse={}
tx_recency = tx_user[['Recency']]
for k in range(1, 10):
    kmeans = KMeans(n_clusters=k, max_iter=1000).fit(tx_recency)
    tx_recency["clusters"] = kmeans.labels_
    sse[k] = kmeans.inertia_
plt.figure()
plt.plot(list(sse.keys()), list(sse.values()))
plt.xlabel("Number of cluster")
plt.show()
```

g2_recency_elbow.py hosted with ❤ by GitHub view raw

Inertia graph:



Here it looks like 3 is the optimal one. Based on business requirements, we can go ahead with less or more clusters. We will be selecting 4 for this example:

```python
#build 4 clusters for recency and add it to dataframe
kmeans = KMeans(n_clusters=4)
kmeans.fit(tx_user[['Recency']])
tx_user['RecencyCluster'] = kmeans.predict(tx_user[['Recency']])

#function for ordering cluster numbers
def order_cluster(cluster_field_name, target_field_name,df,ascending):
    new_cluster_field_name = 'new_' + cluster_field_name
    df_new = df.groupby(cluster_field_name)[target_field_name].mean().reset_
    df_new = df_new.sort_values(by=target_field_name,ascending=ascending).
    df_new['index'] = df_new.index
    df_final = pd.merge(df,df_new[[cluster_field_name,'index']], on=cluster_fie
    df_final = df_final.drop([cluster_field_name],axis=1)
    df_final = df_final.rename(columns={"index":cluster_field_name})
    return df_final

tx_user = order_cluster('RecencyCluster', 'Recency',tx_user,False)
```

g2_recency_cluster.py hosted with ❤ by GitHub view raw

RecencyCluster	count	mean	std	min	25%	50%	75%	max
0	568.0	184.625000	31.753602	132.0	156.75	184.0	211.25	244.0
1	1950.0	17.488205	13.237058	0.0	6.00	16.0	28.00	47.0

We can see how our recency clusters have different characteristics. The customers in Cluster 1 are very recent compared to Cluster 2.

We have added one function to our code which is **order_cluster()**. K-means assigns clusters as numbers but not in an ordered way. We can't say cluster 0 is the worst and cluster 4 is the best. order_cluster() method does this for us and our new dataframe looks much neater:

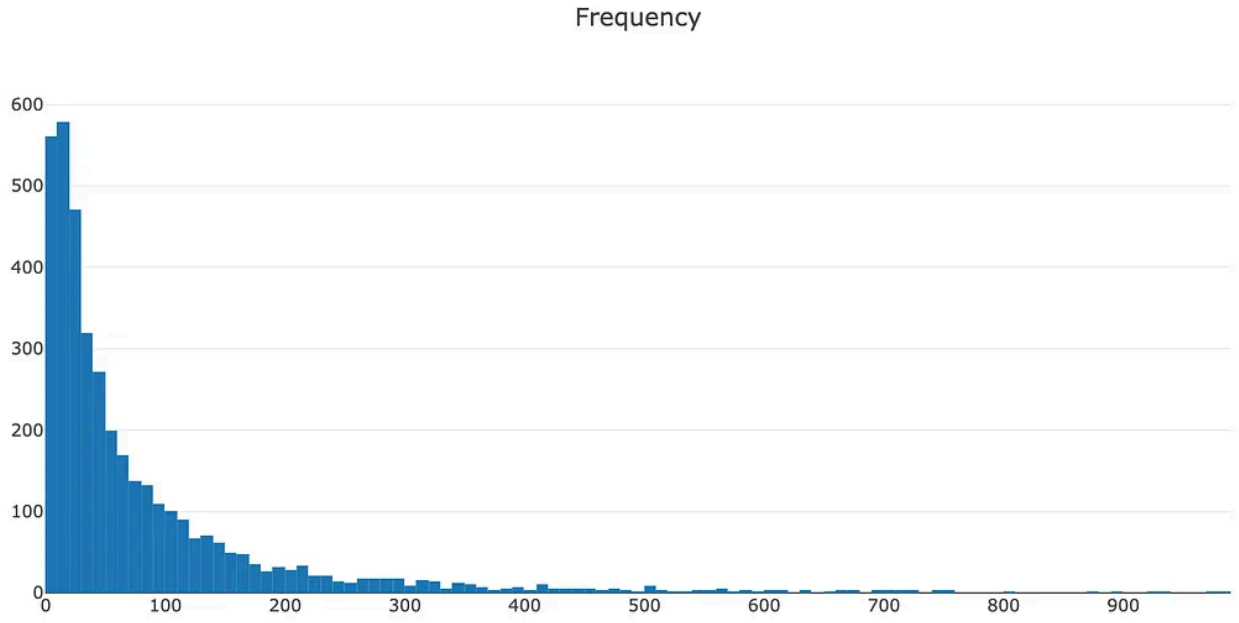
RecencyCluster	count	mean	std	min	25%	50%	75%	max
0	478.0	304.393305	41.183489	245.0	266.25	300.0	336.00	373.0
1	568.0	184.625000	31.753602	132.0	156.75	184.0	211.25	244.0
2	954.0	77.679245	22.850898	48.0	59.00	72.5	93.00	131.0
3	1950.0	17.488205	13.237058	0.0	6.00	16.0	28.00	47.0

Great! 3 covers most recent customers whereas 0 has the most inactive ones.

Let's apply same for Frequency and Revenue.

Frequency

To create frequency clusters, we need to find total number orders for each customer. First calculate this and see how frequency look like in our customer database:



Apply the same logic for having frequency clusters and assign this to each customer:

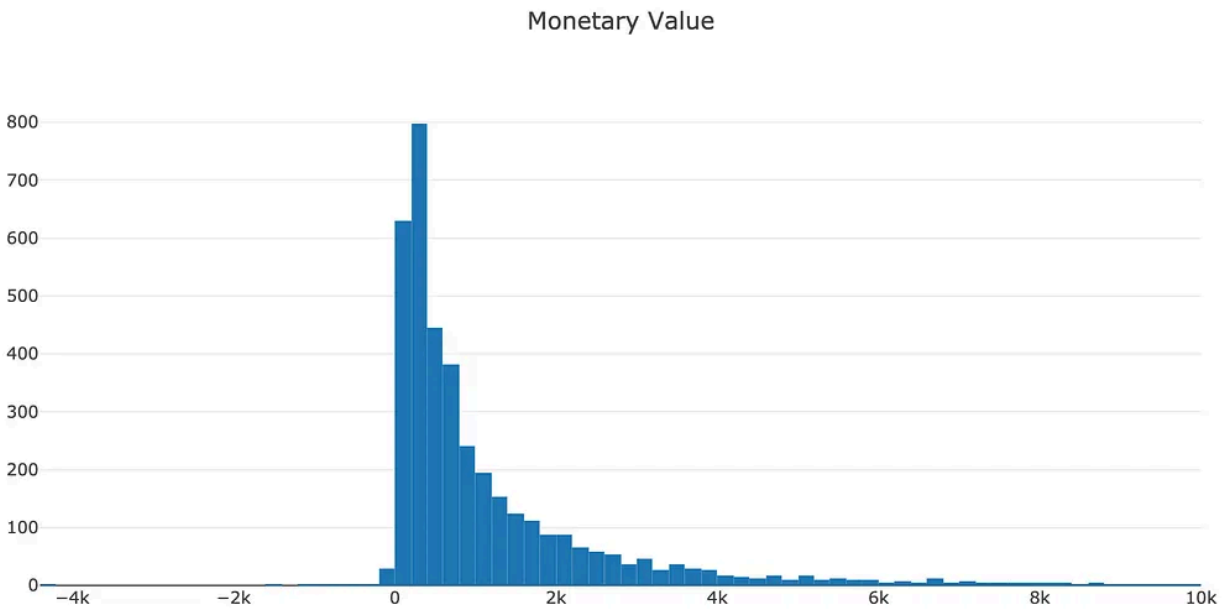
Characteristics of our frequency clusters look like below:

FrequencyCluster	count	mean	std	min	25%	50%	75%	max
0	3496.0	49.525744	44.954212	1.0	15.0	33.0	73.0	190.0
1	429.0	331.221445	133.856510	191.0	228.0	287.0	399.0	803.0
2	22.0	1313.136364	505.934524	872.0	988.5	1140.0	1452.0	2782.0
3	3.0	5917.666667	1805.062418	4642.0	4885.0	5128.0	6555.5	7983.0

As the same notation as recency clusters, high frequency number indicates better customers.

Revenue

Let’s see how our customer database looks like when we cluster them based on revenue. We will calculate revenue for each customer, plot a histogram and apply the same clustering method.



We have some customers with negative revenue as well. Let's continue and apply k-means clustering:

RevenueCluster	count	mean	std	min	25%	50%	75%	max
0	3688.0	908.182672	923.507907	-4287.63	263.3325	572.685	1258.675	4330.67
1	233.0	7775.420687	3638.011093	4345.50	5178.9600	6568.720	9167.820	21535.90
2	27.0	43070.445185	15939.249588	25748.35	28865.4900	36351.420	53489.790	88125.38
3	2.0	221960.330000	48759.481478	187482.17	204721.2500	221960.330	239199.410	256438.49

Overall Score

Awesome! We have scores (cluster numbers) for recency, frequency & revenue. Let’s create an overall score out of them:

```
tx_user.groupby('OverallScore')['Recency','Frequency','Revenue'].mean()
```

OverallScore	Recency	Frequency	Revenue
0	304.584388	21.995781	303.339705
1	185.362989	32.596085	498.087546
2	78.972856	47.060803	871.842586
3	20.662252	68.374172	1089.271213
4	14.892617	271.755034	3607.097114
5	9.662162	373.290541	9136.946014
6	7.740741	876.037037	22777.914815
7	1.857143	1272.714286	103954.025714
8	1.333333	5917.666667	42177.930000

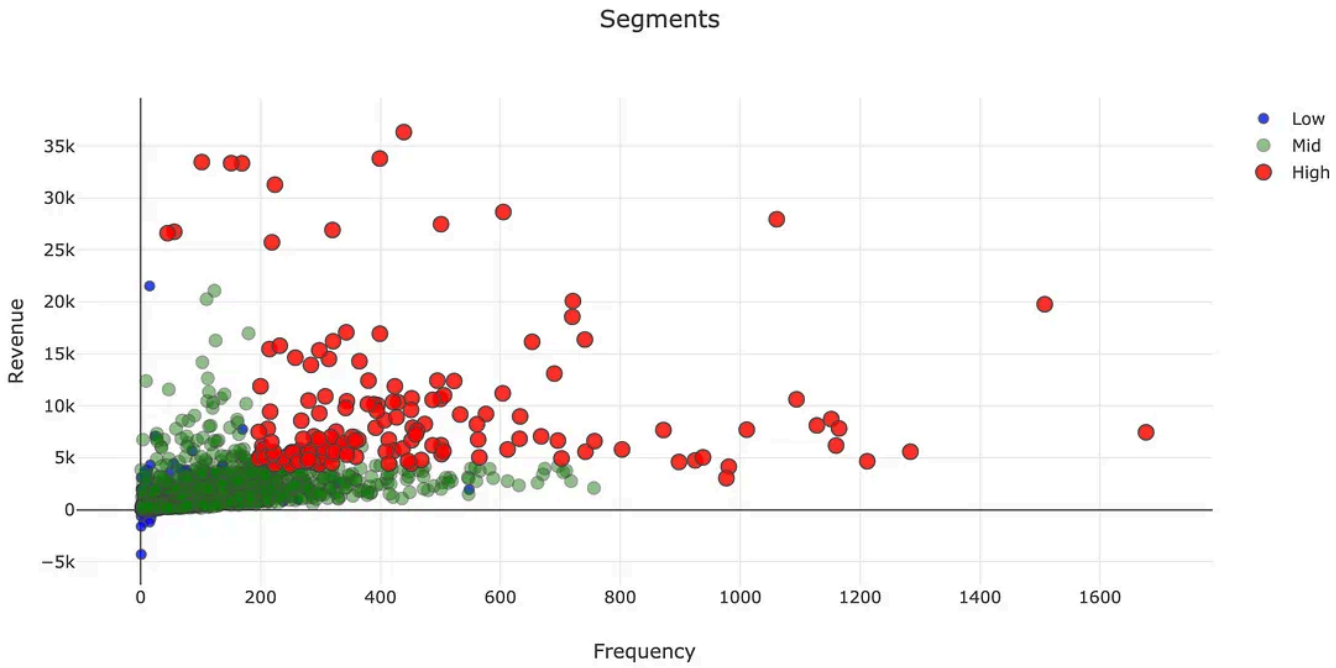
The scoring above clearly shows us that customers with score 8 is our best customers whereas 0 is the worst.

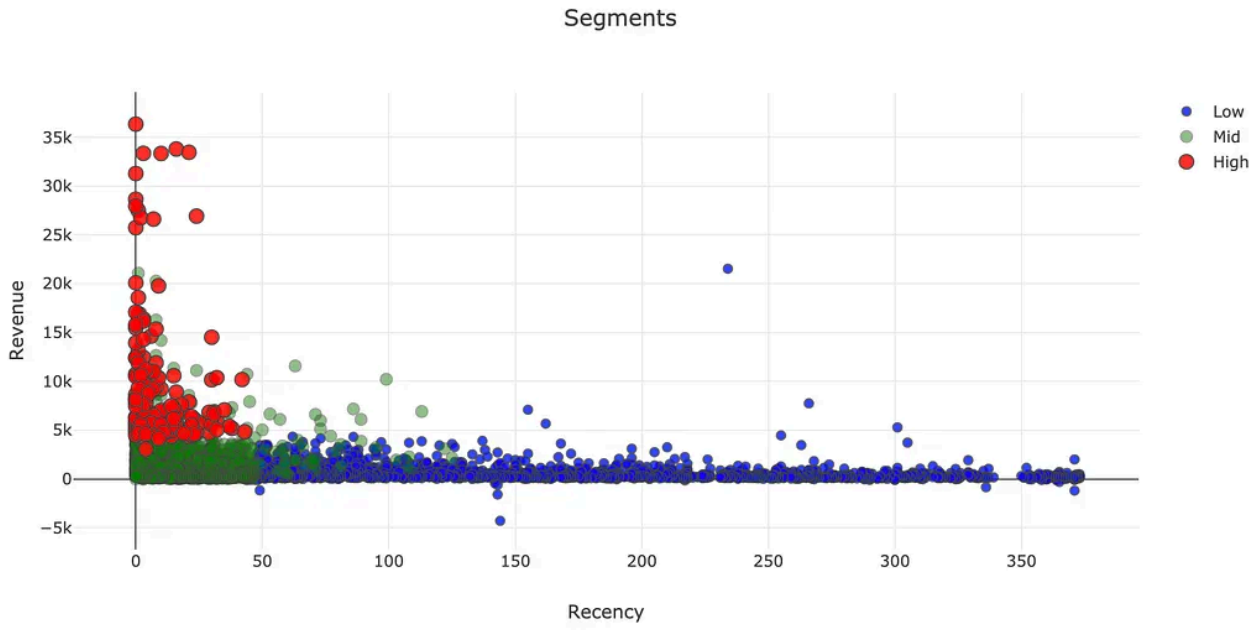
To keep things simple, better we name these scores:

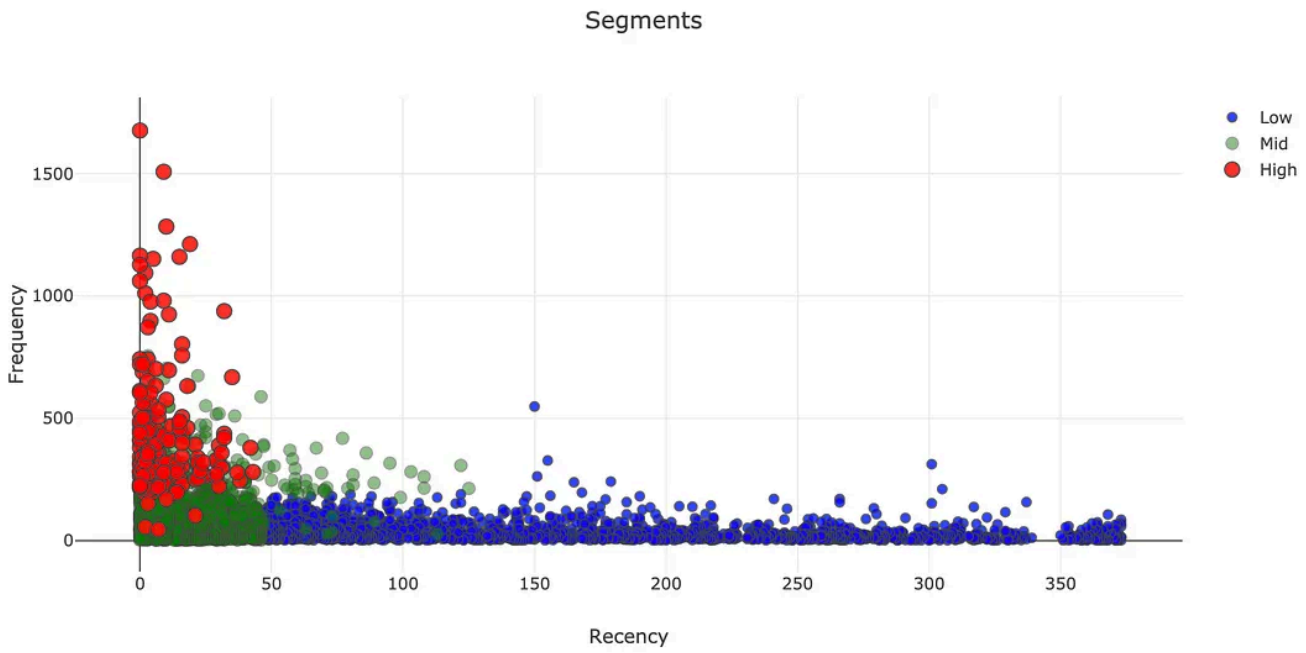
- 0 to 2: Low Value
- 3 to 4: Mid Value
- 5+: High Value

We can easily apply this naming on our dataframe:

Now, it is the best part. Let’s see how our segments distributed on a scatter plot:


Segments
Low
Mid
High
Revenue
35k
30k
25k
20k
15k
10k
5k
0
−5k
0
200
400
600
800
1000
1200
1400
1600
Frequency


Segments
Low
Mid
High
Revenue
35k
30k
25k
20k
15k
10k
5k
0
−5k
0
50
100
150
200
250
300
350
Recency



You can see how the segments are clearly differentiated from each other in terms of RFM. You can find the code snippets for graphs below:

We can start taking actions with this segmentation. The main strategies are quite clear:

- High Value: Improve Retention
- Mid Value: Improve Retention + Increase Frequency
- Low Value: Increase Frequency

Getting more and more exciting! In Part 3, we will be calculating and predicting lifetime value of our customers.

You can find the jupyter notebook for this article here.

*Ideally, what we do here can be easily achieved by using quantiles or simple binning (or Jenks natural breaks optimization to make groups more accurate) but we are using k-means to get familiar with it.

I've started to write the more advanced and updated version of my articles here. Feel free to visit, learn more and support.

Growth Hacker https://www.linkedin.com/in/karamanbaris Schedule a free call with me here: https://app.growthmentor.com/baris-karaman